

ALF - Interactions

Florian Goth

Universität Würzburg
Institut für Theoretische Physik und Astrophysik

15. Feb. 2022

- The interaction Hamiltonian
- Perfect squares
- Hubbard model + The operator type
- Some predefined structures

$$\hat{\mathcal{H}} = \hat{\mathcal{H}}_T + \hat{\mathcal{H}}_V + \hat{\mathcal{H}}_I + \hat{\mathcal{H}}_{0,I}$$

where

$$\hat{\mathcal{H}}_T = \sum_{k=1}^{M_T} \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger T_{xy}^{(ks)} \hat{c}_{y\sigma s} \equiv \sum_{k=1}^{M_T} \hat{T}^{(k)},$$

$$\hat{\mathcal{H}}_V = \sum_{k=1}^{M_V} U_k \left\{ \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \left[\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger V_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) + \alpha_{ks} \right] \right\}^2 \equiv \sum_{k=1}^{M_V} U_k \left(\hat{V}^{(k)} \right)^2,$$

$$\hat{\mathcal{H}}_I = \sum_{k=1}^{M_I} \hat{Z}_k \left(\sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger I_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) \equiv \sum_{k=1}^{M_I} \hat{Z}_k \hat{I}^{(k)}.$$

We will consider the interaction:

$$\hat{\mathcal{H}}_V = \sum_{k=1}^{M_V} U_k \left\{ \sum_{\sigma=1}^{N_{col}} \sum_{s=1}^{N_{fl}} \left[\left(\sum_{x,y}^{N_{dim}} \hat{c}_{x\sigma s}^\dagger V_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) + \alpha_{ks} \right] \right\}^2 \equiv \sum_{k=1}^{M_V} U_k \left(\hat{V}^{(k)} \right)^2,$$

- $V_{xy}^{(ks)}$, interaction matrix
- α_{ks} , the shift
- U_k , the interaction strength per operator
- N_{dim} , spatial volume
- N_{fl} , number of flavours
- N_{col} , number of colours
- M_V , number of interaction terms/perfect squares

And needs to be a sum of perfect squares!

$$V = U \sum_i^{N_L} n_{i\uparrow} n_{i\downarrow}$$

→ Need to bring $n_{i\uparrow} n_{i\downarrow}$ into a square form.

From (Schumann1988, PLA 134, 3) we have the one-parameter representation for the Hubbard interaction (already slightly simplified):

$$n_{i\uparrow} n_{i\downarrow} = \frac{1}{4} - \frac{3\alpha}{4} + R_i^z + (3\alpha - 1)(S_i^z)^2 + (3\alpha + 1)(R_i^z)^2$$

with $S_i^z = \frac{1}{2}(n_{i\uparrow} - n_{i\downarrow})$, $R_i^z = \frac{1}{2}(n_{i\uparrow} + n_{i\downarrow} - 1)$

two choices for the perfect square:

- $\alpha = -\frac{1}{3} : n_{i\uparrow} n_{i\downarrow} = \frac{1}{2}(n_{i\uparrow} + n_{i\downarrow}) - 2(S_i^z)^2$
- $\alpha = \frac{1}{3} : n_{i\uparrow} n_{i\downarrow} = (n_{i\uparrow} + n_{i\downarrow} - 1) + 2(R_i^z)^2$

Let's pretend we are interested in the S_z representation (called M_z in the docs.)

So we have

$$\begin{aligned}
 V &= U \sum_i^{N_L} n_{i\uparrow} n_{i\downarrow} = -2U \sum_i^{N_L} \left(\frac{1}{2} (n_{i\uparrow} - n_{i\downarrow}) \right)^2 = -\frac{U}{2} \sum_i^{N_L} (n_{i\uparrow} - n_{i\downarrow})^2 \\
 &= \sum_i^{N_L} -\frac{U}{2} \left\{ \sum_{s=1}^2 s c_{i,s}^\dagger c_{i,s} \right\}^2
 \end{aligned}$$

Let's compare with the generic form

$$\sum_{k=1}^{M_V} U_k \left\{ \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \left[\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger V_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) + \alpha_{ks} \right] \right\}^2$$

We can read off, the required parameters: $N_{\text{fl}} = 2$, $N_{\text{col}} \equiv N_{\text{SUN}} = 1$, $M_V = N_L$,

$U_k = \frac{U}{2}$, $V_{xy}^{(i,s=1)} = \delta_{x,y} \delta_{x,i}$, $V_{xy}^{(i,s=2)} = -\delta_{x,y} \delta_{x,i}$ and $\alpha_{ks} = 0$

How to write the Fortran code?

- create your hamiltonian submodule in the subfolder Hamiltonians.
- add your Hamiltonian name to Hamiltonians.list.
- provide required data.

```
submodule (Hamiltonian_main) ham_my_ham_name_smod
Use Operator_mod
...
Implicit none
Type, extends(ham_base) :: ham_my_ham_name
contains
  procedure, nopass :: Ham_Set
  procedure, nopass :: Alloc_obs
  procedure, nopass :: Obser
  procedure, nopass :: ObserT
...
end type ham_my_ham_name
Type (Operator), dimension(:,,:), allocatable :: Op_V ! -> base
Type (Operator), dimension(:,,:), allocatable :: Op_T ! -> base
Integer :: Ndim ! -> base
Integer :: N_FL ! -> base
Integer :: N_SUN ! -> base
Integer :: Ltrot
Integer :: L1, L2
real (Kind=Kind(0.d0)) :: Ham_T, ham_U, Ham_chem
...
```

! Operator_mod.F90

Type Operator

...

complex :: O(:, :)

integer :: P(:)

integer :: N

complex :: g, alpha

...

End Type

User visible parts:

- $Op\%N$ effective dimension
- $Op\%O$ complex $N \times N$ matrix $V = P^T O P$
- $Op\%P$ a vector representation of the projector P .
- $Op\%g$ coupling strength $\sqrt{\Delta\tau U^k}$ (HST)
- $Op\%\alpha$ the constant α .

With this information we can recover the interaction matrix. $V = P^T O P$

For the product representation it holds

$$e^V = e^{P^T O P} = 1 + P^T (e^O - 1) P$$

Thereby we mostly have to exponentiate small matrices O .

Our convention: We have a `Ham_V` subroutine that sets up the interaction:

```
Subroutine Ham_V
```

```
...
```

```
Allocate(Op_V(Ndim,N_FL)) ! create array
```

```
! set up storage of elements of array
```

```
do nf = 1,N_FL
```

```
do i = 1, Ndim
```

```
Call Op_make(Op_V(i,nf), 1) ! 1 since the Hubbard interaction has 1 non—vanishing entry
```

```
enddo
```

```
enddo
```

```
Do nf = 1,N_FL ! two flavours
```

```
X = 1.0
```

```
if (nf == 2) X = -1.0
```

```
Do i = 1,Ndim
```

```
Op_V(i,nf)%P(1) = I
```

```
Op_V(i,nf)%O(1,1) = 1.0
```

```
Op_V(i,nf)%g = X*SQRT(DTAU*ham_U/2.0) ! Sqrt due to HST
```

```
Op_V(i,nf)%alpha = 0.0
```

```
Op_V(i,nf)%type = 2
```

```
Call Op_set( Op_V(i,nf) ) ! initialize internal state
```

```
Enddo
```

```
Enddo
```

Then you can do *call Ham_V* in the subroutine `Ham_set`

Hubbard M_z representation

```
Subroutine Ham_V
...
Allocate(Op_V(Ndim,N_FL)) ! create array

do i = 1, Latt%N
  call predefined_Int_U_Mz(Op_V(i, 1), Op_V(i,2), i, dtau, Ham_U )
enddo
```

Hubbard $SU(N)$

```
Subroutine Ham_V
...
Allocate(Op_V(Ndim,N_FL)) ! create array

do i = 1, Latt%N
  call predefined_Int_U_SUN(Op_V(i, 1), I, N_SUN, dtau, Ham_U)
enddo
```

Another model is the t - V model:

$$H = \sum_{i,j,\sigma}^N c_{i,\sigma}^\dagger T_{ij} c_{j\sigma} - \frac{V}{N} \sum_{b=\langle i,j \rangle} \left(\sum_{\sigma=1}^N (c_{i\sigma}^\dagger c_{j\sigma} + c_{j\sigma}^\dagger c_{i\sigma}) \right)^2$$

with $\langle i, j \rangle$ denoting a bond between i, j .

- $N_{dim} = (\text{number of lattice sites}) \times (\text{nr of orbitals})$
- $N_{fl} = 1$
- $N_{col} = N$
- $M_T = 1$
- $U_k = -V/N$
- $M_V = (\text{number of bonds})$
- $\alpha_{ks} = 0$
- $V^k, k = \langle i, j \rangle, O = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, P_1 = i, P_2 = j$

Sign-problem discussed in (FFA and T. Grover, PRX 6, 041049, 2016)
 Predefined: Look for Predefined_Int_V_SUN()

Fermion-Ising coupling

$$\hat{\mathcal{H}}_{V,i,j} = \hat{Z}_{i,j} \xi \sum_{\sigma=1}^{N_{\text{col}}} \left(\hat{c}_{i\sigma}^{\dagger} \hat{c}_{j\sigma} + \hat{c}_{j\sigma}^{\dagger} \hat{c}_{i\sigma} \right),$$

Call `Predefined_Int_Ising_SUN( OP, I, J, DTAU, XI )`

Long-range Coulomb repulsion

$$\hat{\mathcal{H}}_V = \frac{1}{N} \sum_{\vec{i}, \vec{j}} \left(\hat{n}_i - \frac{N}{2} \right) V_{i,j} \left(\hat{n}_j - \frac{N}{2} \right),$$

Call `Predefined_Int_LRC( OP, I, DTAU )`

J_z - J_z interaction

$$\hat{\mathcal{H}}_{V,i,j} = -\frac{|J_z|}{2} (S_i^z - \text{sgn } |J_z| S_j^z)^2 = J_z S_i^z S_j^z - \frac{|J_z|}{2} (S_i^z)^2 - \frac{|J_z|}{2} (S_j^z)^2$$

Call `Predefined_Int_Jz( OP_up, Op_do, I, J, DTAU, Jz )`

$$H_{\text{spin}} + H_{\text{Fermion}} + H_{\text{Kondo}}$$

with

$$H_{\text{Spin}} = \sum_{i,j} J_{ij}^z S_i^z S_j^z + J_{ij}^{\perp} (S_i^+ S_j^+ + h.c.)$$

$$H_{\text{Fermion}} = \sum_{x,y,\sigma} c_{x\sigma}^{\dagger} T_{xy} c_{y\sigma} + U \sum_x (n_{x\downarrow} - \frac{1}{2})(n_{x\uparrow} - \frac{1}{2})$$

$$H_{\text{Kondo}} = \sum_{i,x} J_{i,x}^K \mathbf{c}_x^{\dagger} \left[\frac{\sigma^z}{2} S_i^z - \frac{(-1)^x}{2} (\sigma^+ S_i^- + \sigma^- S_i^+) \right] \mathbf{c}_x$$

Using $\mathbf{S}_i = \frac{1}{2} \mathbf{f}_i^{\dagger} \boldsymbol{\sigma} \mathbf{f}_i$ we obtain the representation in perfect squares

$$\begin{aligned} H = & - \sum_{i,j} |J_{ij}^{\perp}| \left[\frac{1}{2} (\tilde{\mathbf{f}}_i^{\dagger} \tilde{\mathbf{f}}_j + \tilde{\mathbf{f}}_j^{\dagger} \tilde{\mathbf{f}}_i)^2 + \frac{1}{4} \left((\tilde{\mathbf{f}}_i^{\dagger} \tilde{\mathbf{f}}_i - 1) + (\tilde{\mathbf{f}}_j^{\dagger} \tilde{\mathbf{f}}_j - 1) \right)^2 \right] \\ & - \frac{1}{8} \sum_{ij} |J_{ij}^z| \left((\tilde{\mathbf{f}}_i^{\dagger} \tilde{\mathbf{f}}_i - 1) - \frac{J_{ij}^z}{|J_{ij}^z|} (\tilde{\mathbf{f}}_j^{\dagger} \tilde{\mathbf{f}}_j - 1) \right)^2 - U \sum_i (\tilde{\mathbf{f}}_i^{\dagger} \tilde{\mathbf{f}}_i - 1)^2 \\ & + \sum_{x,y} \tilde{\mathbf{c}}_x^{\dagger} T_{xy} \tilde{\mathbf{c}}_y - \frac{U}{2} \sum_x (\tilde{\mathbf{c}}_x^{\dagger} \tilde{\mathbf{c}}_x - 1)^2 - \frac{1}{4} \sum_{i,x} J_{i,x}^K (\tilde{\mathbf{f}}_i^{\dagger} \tilde{\mathbf{c}}_x + \tilde{\mathbf{c}}_x^{\dagger} \tilde{\mathbf{f}}_i)^2 \end{aligned}$$

$\tilde{\mathbf{f}}^{\dagger} = (f_{\uparrow}^{\dagger}, f_{\downarrow}^{\dagger}), \tilde{\mathbf{c}}_x^{\dagger} = (c_{x\uparrow}^{\dagger}, (-1)^x c_{x\downarrow}^{\dagger})$ Further results: T. Sato, FFA, T. Grover PRL, 2018